

QMTester: Program-Level Metamorphic Testing for Quantum Programs

Anonymous Submission

Abstract—A quantum program is more than its final `QuantumCircuit`: it is a builder, parameterized APIs, classical-register mappings, and ancilla protocols, and many real defects enter these program-level contracts before the circuit is emitted—invisible to a metamorphic transform that rewrites only the *final* circuit. QMTester checks *program-level* metamorphic relations over Qiskit by transforming a program input, rebuilding the circuit, and comparing canonicalized counts. Its discipline is *canonicalization-as-admission*: a relation runs only when its output map is a documented bijection on the measured support, so a representation change is undone, not mistaken for a fault, and a register-dropping candidate rejected, not scored; admitted pairs go to a permutation two-sample test. Of five distribution-level families, two—qubit-role permutation and register remapping—and ancilla-uncompute defects crossing a non-unitary boundary need builder semantics: no MorphQ-class circuit transform diverges source from follow-up for a builder-ignores-input defect (§II-C). The other two (QFT round-trip, parameter periodicity) are circuit-expressible, included for coverage; a complementary *static* register-shape check covers the most common builder defect—the redundant `measure_all` register the bijection excludes. Specificity is inclusion-independent: zero false positives over 107 fixed variants and 50 correct programs. Under realistic readout/crosstalk the smallest uncorrected p narrows to 0.0035, but the thin-margin subject’s $m=96$ pairs give $\alpha/m=0.0005$, so the 0/50 survive by correction, not luck, and real-defect detection holds 7/7—though one cross-boundary ancilla variant, its family the most noise-sensitive, trips a false positive there. On ten real defects (seven Bugs4Q, three mined from recent Q&A) it detects 10/10 at 0 false positives (Wilson [0.722, 1.000]); on the seven it matches a reference-oracle ceiling with *no* oracle, a *deployable* static analyzer catches only the syntactic register-shape case, and MorphQ’s 0/7 is builder-layer blindness. An oracle-light ablation shows *canonicalization*, not the statistic, carries detection—a minimal canonicalize-then-equality variant reaches 7/7—while statistics carry specificity and the one probabilistic family (power floor at TVD $\approx$ 0.02–0.03). An exhaustive LintQ screen (33,995 findings) and an OOPSLA’22 corpus find distribution-detectable builder defects rare; the static check covers 37 `measure_all` findings. All numbers regenerate from frozen manifests and run-isolated shards.

Index Terms—quantum software testing, program-level metamorphic testing, builder-level relations, statistical testing, Qiskit

I. INTRODUCTION

Quantum software is increasingly developed with familiar software-engineering workflows, but it is tested under an execution model that makes ordinary test oracles fragile. A quantum program usually returns samples from a distribution rather than a single deterministic value. The full quantum state is not directly observable without changing the computation, and the number of possible states grows exponentially with the number of qubits. Even when the intended circuit is

correct, finite shots and backend noise can move observed frequencies enough to make exact-output assertions misleading. These properties turn regression testing into an oracle problem: developers need fault indicators without writing a fully specified expected distribution for every program and backend configuration [1]–[3].

A point that is easy to miss in this setting is *where* the bug actually lives. A Qiskit program is not just the final `QuantumCircuit` object that reaches the backend. It is a builder that allocates qubits and classical registers, a set of parameterized APIs, a mapping from logical roles to wire indices, a measurement layout, and ancilla protocols that must uncompute cleanly. Many real defects are introduced at this program-contract layer—a permuted control qubit in a multi-controlled gate, a classical register read back in the wrong endianness, a QFT applied in the wrong direction, a rotation angle that breaks an intended periodicity, an ancilla that is never uncomputed—*before* the concrete circuit is emitted. A metamorphic transformation that rewrites the *final* circuit takes the buggy builder output as ground truth and rewrites around it; it therefore cannot expose a bug that is already baked into that output.

Existing work answers different parts of the oracle problem. Bugs4Q provides reproducible quantum-program defects for controlled studies [4]. MorphQ uses metamorphic transformations over generated circuits to expose faults in the Qiskit platform [5]. Differential testing compares quantum software stacks when multiple implementations can run comparable programs [6]. Assertion-based and projective-measurement approaches let developers encode stronger probabilistic or quantum-specific properties when those properties are known [2], [3], [7]. These techniques operate at the *circuit* or *platform* level. We focus on the complementary boundary they leave open: the program/builder contracts that determine the circuit in the first place.

We target that boundary with QMTester, a metamorphic-testing workflow whose relations are defined over *program inputs*, not over a fixed final circuit. Given a program subject—a builder plus the auditable semantics of its inputs—QMTester derives a follow-up *input*, rebuilds the circuit from that follow-up input, executes the source and follow-up circuits under matched settings, canonicalizes the measured count vectors through a validated bijective map, and applies a corrected two-sample test to decide whether the program-level relation is violated. The relation families capture builder-level contracts: qubit-role covariance under input permutation, classical-register remapping, QFT round-trip direction, parameter peri-

odicity, and ancilla uncompute. Each relation is admitted only under explicit, documented preconditions; a candidate that would change the number of measured bits, drop a register, or induce a non-bijective output map is rejected rather than tested.

We are deliberate about scope: we do not claim benchmark dominance over Bugs4Q, and the injected control is not evidence of real-world generality. Our calibrated, auditable claim is that on defects with a documented sound relation, program-level testing *matches a ground-truth reference-oracle differential with no oracle*—not that it beats MorphQ, whose 0/7 is structural blindness to the builder layer, not a comparative loss—at zero false positives on the matched fixed variants. A circuit-level-only diagnostic in our own artifact shows *no* advantage over MorphQ, motivating the move to the program level rather than a larger circuit-level catalog.

The empirical account is traceable: every headline number resolves to a row of a canonical CSV derived from frozen manifests and run-isolated shards, and each included subject’s relation-soundness audit is documented for inspection.

We make three contributions:

- **A program-level instantiation of metamorphic testing for Qiskit.** We argue and demonstrate that a quantum program is a builder, parameterized APIs, register mappings, and ancilla protocols—not only its final circuit—and that defects in these contracts are a distinct target. For a *builder-ignores-input* defect, no circuit-level metamorphic self-transform (the MorphQ class) can produce a source/follow-up divergence—it rewrites the already-emitted buggy circuit rather than re-invoking the builder on a different input—so detection requires rebuilding from a follow-up input (Proposition 1); this necessity holds structurally for qubit-role permutation and classical-register remapping, and across the non-unitary boundary (§II-C) for ancilla-uncompute, whereas `program_qft_round_trip` and `program_parameter_periodicity` are circuit-expressible and included for coverage.
- **Canonicalization-as-admission, in five distribution families plus a complementary static check.** The technical core is an admission discipline: a relation is tested only when its output map is a *documented bijection on the measured support*, which separates a representation change from a logical divergence—candidates that drop a register or yield a non-bijective map are rejected, not scored. We instantiate it in five `program_*` families (`input_permutation`, `classical_remap`, `qft_round_trip`, `parameter_periodicity`, `ancilla_uncompute`) under a quantum-aware period check (a 2π shift is rejected on a controlled rotation, whose period is 4π), with a *complementary static program_register_shape* check—needing no statistics—that brings the prevalent `measure_all` register-shape defect in scope.

- **A prevalence finding (first-class), and an inclusion-independent evaluation.** An exhaustive screen of two real-defect corpora characterizes the niche precisely: distribution-detectable builder defects are *rare*—the most common builder defect is the `measure_all` register-shape change (37 of LintQ’s 33,995 findings), caught by the static register-shape check—so with all 32 build-and-run Bugs4Q bugs audited (9 in scope), we report a real-but-small niche rather than overstate it. Detection on the in-scope defects is *deterministic* for all four real families (TVD = 1, the real QFT subject included) and holds 7/7 under a realistic readout/crosstalk noise model; the statistics are load-bearing for specificity and the one probabilistic family, not for detection. The inclusion-*independent* evidence is specificity (0/107 fixed variants, 0/50 correct programs; smallest per-pair $p = 0.0035$ under the realistic model, Bonferroni still 0/50) and a detectability-unfiltered power curve (floor near TVD 0.025), plus an oracle-light ablation.

II. BACKGROUND AND RELATED WORK

A. Quantum Testing as an Oracle Problem

Metamorphic testing replaces unavailable expected-output oracles with relations between executions of a program [8]–[11]. Prior work reports that relation diversity can alleviate oracle weakness, while still depending on the quality of the chosen relations [12]. For quantum software, this idea is natural because many behavioral expectations are relational: equivalent circuits should induce the same measurement distribution, a logical qubit permutation should not change results after the output order is restored, and a global phase should not affect measurement probabilities. The difficulty is that these statements hold only under explicit preconditions.

The second difficulty is statistical. A source and follow-up execution can produce different empirical count vectors even when their true distributions are equal, while a real fault may produce a small distribution shift that is invisible under a low shot budget. Treating raw count equality as the oracle creates false alarms, while treating every p-value in isolation ignores the multiple follow-up executions generated per source program. We therefore combine relation validity with a corrected two-sample decision rule for categorical count data.

B. Count Vectors and Output Canonicalization

A run of a quantum program produces a *count vector*: a map from measured bitstrings to observed frequencies over n shots. Comparing two runs is comparing two such vectors over a shared support. Two subtleties make this comparison non-trivial.

First, a valid relation often changes the *representation* of the output without changing its logical content. Permuting input qubit roles, remapping a classical register, or reversing a QFT relabels which physical bit carries which logical value. Before any statistical test, QMTester rewrites the follow-up count vector into the source’s logical order using a *canonicalization*

map c induced by the transformation. A relation is admitted only when c is a documented *bijection* on the measured support: a candidate that drops a register, changes the number of measured bits, introduces a measurement where none existed, or yields a non-bijective map is rejected as a coverage loss, not scored as a test outcome.

Second, Qiskit exposes multi-register outputs as space-separated keys with register-dependent bit order and endianness. Canonicalization first flattens these keys and validates their length, then applies the relation’s bijective remap. This is where representation bugs and genuine logical bugs are separated: if two logically equivalent program inputs disagree after a validated bijective remap, the disagreement is a candidate fault rather than a bookkeeping artifact.

C. Non-Unitary Boundaries

Mid-circuit measurement, classical conditionals, reset, and other non-unitary operations break the simple “rewrite-anywhere” assumption that unitary metamorphic relations rely on. A transformation that is sound for a purely unitary prefix can be invalid once it crosses a measurement or a conditioned reset, because the operation’s effect depends on a classical value produced at run time. Program-level relations must therefore declare, per family, whether and how they admit transformations across such boundaries. Several of our relations (e.g. ancilla uncompute and teleportation feedback) are specifically about getting these boundaries right, which is why they are expressed over the program’s builder semantics rather than over the emitted circuit.

D. Positioning

We position QMTester by oracle assumption, evidence model, and *testing level* in Table I. MorphQ is the closest baseline because it also uses metamorphic testing in the Qiskit ecosystem. The decisive distinction is the level at which relations are defined: MorphQ is a *circuit-level* baseline that transforms a generated or given final circuit, whereas QMTester defines relations over *program inputs* and rebuilds the circuit from the follow-up input. This is not a statement that MorphQ is weak. It means MorphQ structurally cannot exploit builder-level semantics: when a defect is already present in the final circuit a builder produced, a circuit-level transformation rewrites a faithful copy of the buggy output and observes no inconsistency. A MorphQ score of 0/7 on our audited program-level subset is therefore an inherent consequence of the testing level, not an unfair comparison or a tuning artifact.

Closest to our setting is the quantum metamorphic-testing line itself. Abreu et al. [17] and Costa et al. [18] develop metamorphic relations for oracle and Shor-style quantum programs, while MorphQ [5] and its reproducibility follow-up MorphQ++ [19] apply metamorphic transformations to the Qiskit quantum platform. These works define relations primarily over the *circuit* (or over an algorithm-level specification) and transform an already-emitted circuit. QMTester differs on two axes that matter for builder-layer defects: (i) its relations are defined over *program/builder inputs* and the

follow-up circuit is *rebuilt* from the transformed input rather than rewritten from the source circuit, and (ii) a candidate is admitted only when its output canonicalization is a validated bijection on the measured support. The first axis is what lets a relation see a defect the buggy builder bakes into its circuit; the second is what prevents a representation change from being scored as a fault. We retain MorphQ as the matched circuit-level baseline because it is the most directly runnable tool on the same Qiskit subjects; the contrast with these algorithm/circuit-level relations is about the testing *level*, not tool quality.

The comparison also clarifies what we do not claim. We do not replace proof-oriented formal methods for programs with available specifications, and we do not make Pauli-string or projective assertions unnecessary when such specifications are known. Quantum Hoare logics, relational logics, embedded circuit languages, and circuit-equivalence checkers offer stronger reasoning obligations when their models and specifications apply [20]–[23]. We use those ideas as a boundary: QMTester checks empirically whether admitted source/follow-up program pairs behave consistently, but it is not a platform differential-testing campaign, a mutation-adequacy metric, a proof system, or a device-backed study. We also do not evaluate QMutPy in this submission; mutation testing assumes an existing test suite and is not a substitute for the oracle-light, builder-level use case we target.

III. QMTESER APPROACH

We design QMTester as a paired-execution workflow over *program inputs*. The input is not a fixed circuit but a *program subject*: a Qiskit builder together with the auditable semantics of its inputs (qubit roles, classical-register layout, parameters, ancilla protocol), a backend configuration, a shot budget, and a set of enabled program-level relation families. The output is a set of relation-level decisions plus a source-level detection decision after multiple-testing correction. Figure 1 summarizes the workflow.

A. Two-Layer Design

QMTester is organized as two layers with a clean interface between them.

Layer 1: relation admission and canonicalization. This layer is subject-agnostic. It admits a candidate transformation only under its declared preconditions, builds the source and follow-up circuits, executes them under matched settings, and canonicalizes the resulting count vectors through a validated bijective map before any test is run (Sec. II-B). A candidate whose canonicalization map is not bijective on the measured support, or that changes the measured-bit count, drops a register, or crosses a non-unitary boundary a family does not admit, is rejected and logged as a coverage loss rather than scored.

Layer 2: program-level subject adapters and relation families. This layer encodes the auditable builder-level semantics of each subject. A `ProgramSubject` adapter exposes how to construct the circuit from a given input and what each

TABLE I

POSITIONING AGAINST RELATED QUANTUM TESTING WORK. THE DECISIVE AXIS IS THE **TESTING LEVEL**: QMTES-TER DEFINES RELATIONS OVER *program/builder inputs*, WHEREAS MORPHQ AND RELATED METAMORPHIC TOOLS TRANSFORM A *final circuit*.

Work	Testing level	Primary focus	Oracle / evidence model	Relationship to QMTesTer
QMTesTer (ours)	Program / builder	Metamorphic testing of Qiskit program inputs	Program-relation violation via canonicalized count-vector test	—
MorphQ [5]	Circuit / platform	Metamorphic testing of the Qiskit platform	Generated final-circuit pairs and platform-level failures	Closest baseline; circuit-level, so it cannot reach builder-layer contracts (hence 0 on the audited program-level subset).
Bugs4Q [4]	— (benchmark)	Reproducible quantum-program bugs	Curated bug/fix pairs	Real-defect substrate; we use an audited program-level subset with documented relations.
QDiff [6]	Stack	Differential testing across stacks	Cross-stack output discrepancies	Complementary; needs comparable implementations across stacks.
QMutPy [13]	Circuit / suite	Mutation testing for Qiskit	Existing tests kill or survive mutants	Related only; <i>not evaluated</i> here—assumes a test suite, not an oracle-light baseline.
QOPS/SB-QOPS [14], [15]	Circuit	Pauli-string and search-based testing	Measurement-centric Pauli-string oracles	Strong oracle-centered alternative; targets settings with Pauli specifications.
QEMI [16]	Stack	EMI-style stack testing	Dead-code variants should preserve behavior	Complementary metamorphic direction over dead-code variants.
Statistical assertions [2]	Program	Probabilistic program assertions	Developer-specified probability properties	Statistical discipline but requires absolute probability specifications.
Projective-measurement testing [3], [7]	Circuit	Efficient assertion checking	Projective predicates and reduced measurements	Complementary route to stronger or cheaper relation-specific oracles.

input position means (which qubit plays which role, which classical bit records which logical value, which parameter must respect which periodicity, which register is an ancilla). A `ProgramRelationCandidate` pairs a subject with a relation family and produces the follow-up *input* together with the induced canonicalization map. Because the relation is expressed over inputs and the circuit is *rebuilt* from the follow-up input, a defect that a buggy builder bakes into its output is exposed as a divergence between the source and follow-up outputs.

B. Workflow

We split the workflow into five stages (Fig. 1’s boxes further split stage one into input/validate and stage five into canonicalize/oracle/decision). Stage one normalizes the source subject metadata so that qubit roles, classical registers, measurement layout, and output-bit order are explicit. Stage two enumerates candidate follow-up inputs from the enabled relation families. Stage three checks each candidate against its relation-specific preconditions and constructs the induced canonicalization map before admitting a follow-up. Stage four rebuilds and executes the source and follow-up circuits under the same backend, shot budget, seed policy, and timeout policy. Stage five canonicalizes the observed count vectors and compares them with the statistical oracle.

We make the procedure intentionally conservative about invalid relations. A follow-up input that cannot be justified under the subject’s role semantics, parameter domain, classical-control behavior, or canonicalization bijectivity is rejected before execution. This rejection reduces relation coverage, but it prevents QMTesTer from treating a self-inflicted semantic change as a program fault.

C. Formal Model and Assumptions

We use the formal model to mark exactly where QMTesTer relies on relation validity and where it relies on statistical

Algorithm 1 QMTesTer program-level detection pipeline

Require: subjects $\mathcal{S}$, program relations $\mathcal{R}$, backend b , shots n , seeds, timeout τ , threshold α
Ensure: pair logs, source decisions, rejected relation records

```

1: for all  $P \in \mathcal{S}$  do
2:    $Pairs \leftarrow []$ ;  $Rejects \leftarrow []$ 
3:   for all  $R \in \mathcal{R}$  do
4:     for all  $\iota' \in \text{FOLLOWUPINPUTS}(R, P)$  do
5:       if  $\text{ADMISSIBLE}(P, R, \iota')$  then
6:          $c \leftarrow \text{CANONMAP}(P, R, \iota')$ 
7:          $Pairs \leftarrow Pairs \cup \{(R, P, \text{BUILD}(P, \iota'), c)\}$ 
8:       else
9:          $Rejects \leftarrow Rejects \cup \{(P, R, \iota', \text{REASON}(P, R, \iota'))\}$ 
10:   $Tests \leftarrow []$ 
11:  for all  $(R, P, P', c) \in Pairs$  do
12:     $x \leftarrow \text{EXECUTE}(P, b, n, seeds, \tau)$ 
13:     $y \leftarrow \text{EXECUTE}(P', b, n, seeds, \tau)$ 
14:     $(x, y) \leftarrow \text{CANONICALIZE}(x, y, c)$ 
15:     $(p, sparse) \leftarrow \text{TWO SAMPLE TEST}(x, y)$ 
16:    if  $sparse$  then
17:       $p \leftarrow \text{EXACTORRESAMPLE}(x, y)$ 
18:       $Tests \leftarrow Tests \cup \{(R, p, sparse)\}$ 
19:   $\alpha_{\text{eff}} \leftarrow \alpha / |Tests|$ 
20:   $detected \leftarrow \exists (R, p, s) \in Tests : p < \alpha_{\text{eff}}$ 
21:   $\text{WRITELOGS}(P, Pairs, Tests, Rejects)$ 
22:   $\text{WRITEDECISION}(P, detected, \alpha_{\text{eff}})$ 

```

testing. Let P be a program subject with input ι that builds a circuit $\text{Build}(P, \iota)$ whose measurement outcomes range over bitstrings $B(P)$. A program relation family R induces candidate follow-up inputs ι' ; the predicate $A_R(P, \iota')$ records whether the relation’s preconditions are satisfied and whether the induced map $c_R : B(\text{Build}(P, \iota')) \rightarrow B(\text{Build}(P, \iota))$ is bijective on the measured support. If $A_R(P, \iota')$ holds, QMTesTer admits the follow-up circuit $P' = \text{Build}(P, \iota')$ and records c_R . Under a fixed execution setting e , repeated executions yield empirical count vectors $\hat{D}_{P,e}$ and $\hat{D}_{P',e}$ that estimate the corresponding measurement distributions defined by the circuit semantics and Born rule [24].

We deliberately separate relation obligations from statistical decisions. When the admitted relation is valid for P and the builder implements the intended semantics, then

QMTester workflow

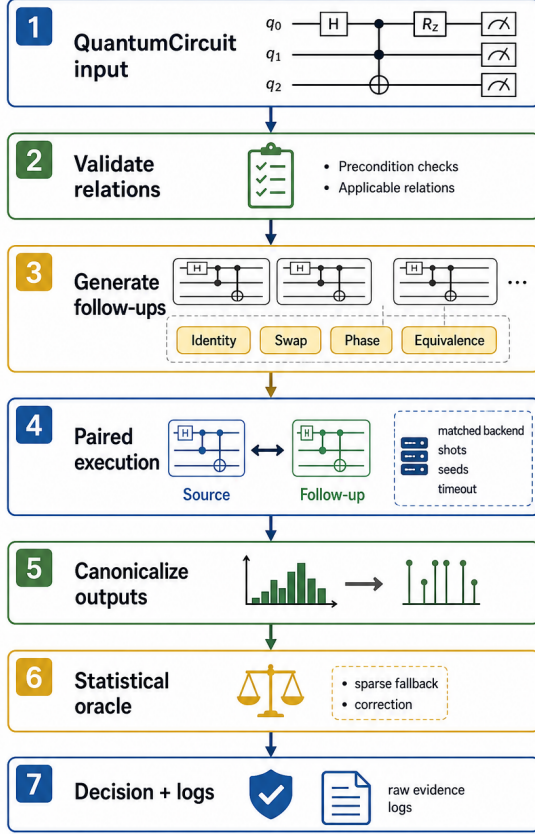


Fig. 1. QMTester workflow: a program subject yields source and follow-up inputs; the builder produces source and follow-up circuits; paired execution yields count vectors; canonicalization validates and remaps outputs; a statistical oracle flags relation violations.

$D_{P,e}(b) = D_{P',e}(c_R^{-1}(b))$ for every observable bitstring b after canonicalization. This is a conditional obligation, not an implementation-level proof; the artifact exposes admitted and rejected follow-up inputs so readers can audit A_R on concrete subjects. For each admitted pair, QMTester tests $H_0 : D_{P,e} = c_R(D_{P',e})$ against a distributional alternative.

Remark 1 (source-level Bonferroni control). For one source subject with m admitted follow-up pairs, rejecting when any pair has $p_i < \alpha/m$ controls the source-level false-alarm probability at α when all m nulls hold: each valid null has $\Pr[p_i < \alpha/m] \leq \alpha/m$, and the union bound gives $\Pr[\exists i : p_i < \alpha/m] \leq \alpha$. This is the standard Bonferroni bound; its only non-trivial content is the premise that the per-pair nulls are *valid*, which is exactly what the empirical type-I check of Sec. IV-E probes (and finds conservative rather than uniform). The bound covers neither invalid relations nor unaudited raw logs.

D. Fault Model and Scope

We target faults that manifest as changes in the measured output distribution under a transformation of program

inputs that should preserve observable behavior. Examples include permuted qubit roles in multi-controlled gates, wrong classical-bit indexing or endianness, inverted QFT direction, parameter values that break an intended periodicity, and ancillas that are not uncomputed. Such faults are common in quantum-program benchmarks because small builder-level mistakes can leave code type-correct while changing the program’s observable behavior.

We do not claim coverage of all quantum bugs. A fault that preserves the measured distribution for the selected input and relation family will not be detected. A fault whose effect size is smaller than the available shot budget may also evade the statistical oracle. Faults that require semantic knowledge outside the program, such as an application-specific cost-function expectation, may need assertions or domain-specific relations. These boundaries define the method’s scope rather than exceptions discovered after evaluation.

E. Program-Level Metamorphic Relations

We organize follow-up generation into five program-level relation families, each defined over program inputs and each carrying a documented soundness argument and a bijective canonicalization map.

- **program_input_permutation:** permute the logical roles of input qubits in a way the subject’s semantics declare covariant, and restore the output order by the inverse permutation. Detection does *not* rest on symmetric-control invariance (which a correct builder satisfies trivially): in our `ccx_role` subject the buggy builder hard-codes `ccx(0, 1, 2)` (target on wire 2) and ignores the requested role map, so permuting the input roles moves the state-prep gates but not the hard-coded `ccx`, and the canonicalized source/follow-up diverge; the role-covariant fix (`ccx(2, 1, 0)`) moves the gate too and stays equal, raising no false positive.
- **program_classical_remap:** remap the classical-register layout (order/endianness) under which results are recorded, then canonicalize back to the source layout. Targets measurement-order and endianness contracts.
- **program_qft_round_trip:** compose the program’s QFT stage with its declared inverse so the round trip is the identity on the logical input. Targets QFT-direction and rotation-sign defects.
- **program_parameter_periodicity:** shift a rotation parameter by a full intended period so the observable distribution is unchanged. The admission check is period-aware: a 2π shift is sound for an uncontrolled Pauli rotation (the extra -1 is a global phase) but a *controlled* Pauli rotation has period 4π (the -1 becomes relative), so a 2π declaration on a controlled rotation is rejected as an invalid context rather than tested—a confusion documented in production Qiskit [25].
- **program_ancilla_uncompute:** require that ancilla and feedback registers return to a clean state, so adding or completing the declared uncompute leaves the logical

output invariant. Targets missing/incorrect uncompute, teleportation feedback, and Grover ancilla protocols.

- `program_register_shape`: a *complementary static check*, not a distribution relation—a builder asked to report in its declared n -bit register must emit that shape, which the prevalent `measure_all` defect violates by doubling the output to $2n$ bits. Needing no statistics, it is the one case a deployable static analyzer also reaches (Table VI); it brings `qiskit_github_12` and the 37 LintQ `measure_all` findings (§V-B) in scope.

Our design goal is distinct builder-level contracts, not a universal relation. We distinguish families by *why* circuit-level testing misses them. `program_input_permutation`, `program_classical_remap`, and largely `program_ancilla_uncompute` are *structurally* builder-only: no circuit-level metamorphic *self*-transform recovers the qubit-role mapping, register layout/endianness, or ancilla identity, because it rewrites the emitted circuit rather than rebuilding from a different input. By contrast `program_parameter_periodicity`, and `program_qft_round_trip` where the QFT block is identifiable, are *circuit-expressible*: a period shift is a local gate edit a circuit-level tool could in principle apply, so MorphQ’s 0 there reflects a *catalog gap*, not structural invisibility—the necessity claim holds strictly for the structurally builder-only families, which Proposition 1 formalizes. (The one mined QPE subject is *incidentally* flagged by a circuit-level swap rewrite—perturbing its wiring, not recovering the role map—so the structural argument is intact.)

Proposition 1 (Necessity for builder-ignores-input defects).

Let a *self-transform* be any MorphQ-class circuit transform—identity insertion, equivalence-preserving relabeling, phase or equivalence rewriting—that rewrites the emitted circuit $C = \text{Build}(P, \iota)$ into a C' preserving its unitary, hence its distribution ($D_{C'} = D_C$). For a *builder-ignores-input* defect (e.g. `ccx_role`), where the buggy builder hard-codes gates independently of ι , no self-transform yields a source/follow-up divergence: it never re-invokes the builder, so the pair agrees by construction. Exposing the defect requires *rebuilding* from a different input ι' —the input-level relation A_R . For ancilla-uncompute the obstruction is the same, the missing uncompute crossing the non-unitary boundary (§II-C) an equivalence-preserving rewrite may not cross. This is *not* the false claim that information is lost on emission: a reference-oracle differential recovers all seven (Table VI) by comparing C against a *different* correct circuit—what no self-transform synthesizes from C alone, but rebuilding from ι' supplies oracle-free. Table II is part of the method: each row implies negative tests for the generator (e.g. `program_classical_remap` must reject non-bijective maps). A purely circuit-level catalog is retained only as a diagnostic baseline (§IV-G), where it shows no advantage over MorphQ.

F. Statistical Oracle

After output canonicalization, we treat observed shot counts from P and admitted follow-up P' as samples from categorical distributions D_P and $D_{P'}$ over the aligned support $B(P)$, testing $H_0 : D_P = D_{P'}$ against the alternative that the distributions differ. *The oracle is a Monte-Carlo permutation two-sample test*: we draw $B = 10,000$ relabelings of the pooled shots and read the p-value off the permutation reference (§IV-E; sparse-cell mechanics below). An asymptotic χ^2 fast path exists for dense supports but is not the operative branch on our exponentially sparse quantum outputs (zero cell-merges across all admitted pairs; the permutation branch decides $\geq 85.7\%$ of pairs), so it is reported, not relied on. The permutation statistic is the pooled-denominator two-sample form

$$T = \sum_{k=1}^K \frac{(x_k - e_k)^2 + (y_k - f_k)^2}{e_k + f_k},$$

$$e_k = \frac{n_x(x_k + y_k)}{n_x + n_y}, \quad f_k = \frac{n_y(x_k + y_k)}{n_x + n_y},$$

for source/follow-up shot totals n_x, n_y . T is a monotone transform of the standard Pearson two-sample statistic [26]—one-half of it under matched totals $n_x = n_y$ —so the permutation reference, which decides the vast majority of pairs, is unaffected by the choice. (The $\chi^2_{K'-1}$ fast path acts on the *standard* Pearson statistic; per above it is untaken here.)

Sparse-cell handling. Quantum outputs are sparse because the support $\{0, 1\}^q$ grows exponentially with the qubit count q . The chi-squared approximation degrades when too many cells have small expected counts. We use a two-stage rule with explicit thresholds rather than a single heuristic:

- 1) *Cell merging*. Sort cells by e_k ascending. Merge any cell with $e_k < 5$ into a single “other” bin (Cochran’s rule). If after merging the number of effective bins K' satisfies $K' \geq 5$ and at least 80% of bins have $e_k \geq 5$, we accept the asymptotic chi-squared p-value.
- 2) *Exact / resampling fallback*. Otherwise we mark the pair `sparse=true` and use a Monte-Carlo permutation test on the pooled multinomial: we draw $B = 10,000$ permutations of the $n_x + n_y$ pooled shots into two groups of size n_x and n_y , recompute χ^2 , and report $\hat{p} = (1 + \#\{b : \chi_b^2 \geq \chi_{\text{obs}}^2\}) / (B + 1)$. The branch is logged per pair so readers can recompute either from raw counts.

As noted above, the fast path is not the operative branch on our corpus (zero cell-merges; $K' = K$ throughout), and the permutation branch decides $\geq 85.7\%$ of pairs (§IV-E).

Multiple-testing correction. QMTester applies Bonferroni correction over all admitted follow-up pairs generated for the same source subject. With m pairs and family-wise threshold $\alpha = 0.05$, the default per-pair threshold is α/m ; step-down Holm correction is reported as a sensitivity check but is not mixed with the default decision rule [27]. We use the source subject rather than the whole benchmark as the correction unit. This choice controls the chance that one subject is flagged

TABLE II

PROGRAM-LEVEL RELATION-VALIDITY OBLIGATIONS FOR QMTES-TER. EACH RELATION IS DEFINED OVER *program inputs*; THE FOLLOW-UP CIRCUIT IS REBUILT FROM THE FOLLOW-UP INPUT AND ITS OUTPUTS ARE REMAPPED THROUGH A VALIDATED BIJECTIVE CANONICALIZATION MAP BEFORE TESTING.

Relation family	Preconditions	Invariant	Target bug classes	Invalid contexts	Implementation checks
<code>program_input_permutation</code>	Subject declares a covariant permutation of input qubit roles (e.g. symmetric controls); inverse permutation restores output order.	Canonicalized bitstring distribution matches the source after the inverse role permutation.	Qubit-role bookkeeping in multi-controlled and symmetric constructions.	Permutations the subject does not declare covariant; untracked classical side effects.	Validate role permutation, bijective inverse, and classical-bit remapping.
<code>program_classical_register</code>	Classical-register layout (order/endianness) is remappable and canonically invertible to the source layout.	Logical outcome distribution is invariant after remapping classical bits to source order.	Measurement-order and endianness contracts.	Remaps that drop or duplicate classical bits, or change the measured-bit count.	Validate bijective register map, key length, and flatten/endianness handling.
<code>program_qft_round_trip</code>	Subject exposes a QFT stage with a declared inverse over the same ordered qubits.	QFT composed with its inverse acts as the identity on the logical input.	QFT-direction and rotation-sign defects.	Approximate or truncated QFT without a matching inverse; mismatched qubit order.	Check QFT/inverse pairing, qubit order, and round-trip identity.
<code>program_parameter_periodicity</code>	Rotation parameter has a declared period T ; shifting by T preserves the observable.	Measurement distribution is invariant under a full-period parameter shift.	Angle handling and periodicity contracts (also drives hard mutants).	Parameters whose period is not declared or is broken by the subject; non-periodic gates.	Confirm declared period and reject shifts outside the periodic domain.
<code>program_ancilla_uncompute</code>	Ancilla-feedback registers are declared and must return to a clean state.	Completing the declared uncompute leaves the logical output invariant.	Missing/incorrect uncompute, teleportation feedback, Grover ancilla protocols.	Transformations that leave an ancilla entangled or cross a non-unitary boundary the family does not admit.	Verify ancilla cleanliness, feedback handling, and uncompute completion.

because many follow-up variants were tried, while avoiding an overly conservative threshold that couples unrelated subjects.

Noisy-simulator calibration. On noisy backends, two executions of the *same* source circuit under independent seeds can produce statistically distinguishable count vectors purely from backend variability. We separate that variability from a real follow-up deviation by executing P under $K = 20$ independent seeds at shot budget n , computing the $\binom{K}{2} = 190$ pairwise null statistics after the same canonicalization and sparse-cell rule, and using the $(1 - \alpha/m)$ -quantile as the calibrated rejection threshold $\hat{\tau}_P$ for that subject. The calibration runs are logged separately so readers can recompute $\hat{\tau}_P$ from the 20 raw count vectors. This procedure is a pragmatic complement to learned noise-mitigation methods [28] and does not require a noise model.

We report all detection and false-positive rates with Wilson 95% confidence intervals [29], and the artifact records, for every admitted pair, the raw count vectors, χ^2 statistic, asymptotic p-value, sparse-cell decision, fallback p-value (if applicable), and Bonferroni-corrected threshold. The prototype is a Python workflow over Qiskit’s `QuantumCircuit`; all reported claims are simulator-based, and device evaluation is out of scope (Sec. V-B).

IV. EVALUATION

We organize the evaluation around four program-level research questions—RQ1 (audited Bugs4Q defects), RQ2 (balanced injected positive control), RQ3 (false positives), and RQ4 (sensitivity floor)—plus a circuit-level-only diagnostic run (§IV-G). Each claim is tied to a dataset, baseline, and metric, and each dataset is introduced with its research question below. MorphQ is the only matched baseline; QMutPy is not evaluated (it assumes an existing test suite), and stack/assertion tools are related-only comparators (Table I). The primary

evidence is program-level; the circuit-level run is retained only as a diagnostic and is not used as a main claim.

A. Execution and Evidence Pipeline

All runs use Qiskit Aer under a pinned environment, a fixed master seed, a default shot budget for detection, and a per-pair wall-clock timeout. MorphQ is run on the same subjects under identical backends, shot budgets, seeds, and timeouts. Every headline number resolves to one row of a canonical CSV that is derived by `derive_metrics.py` from run-isolated summary shards under a frozen run directory; the derivation enforces expected denominators, rejects missing or stale-mixed rows, and re-derives the canonical CSV to a byte-identical copy (*canonical integrity check: PASS*). Table III is the canonical view.

B. RQ1: Real Program-Level Defects

How many audited Bugs4Q program-level defects does QMTes-ter detect?

We *included* a Bugs4Q subject only when a sound program-level relation could be justified (an input transform, a bijective canonicalization map, and a soundness argument tied to builder semantics). The denominators form one *closed* chain: of the 42 Bugs4Q bugs [4], 32 build and run under our pinned Qiskit (the diagnostic set, §IV-G); we audited *all* 32 (per-bug dispositions in the artifact), of which 9 admit a sound builder-contract relation and 23 are out of scope (12 api-misuse, the rest output-formatting, platform/runtime, or wrong-gate; Table V). Seven of the nine are runnable distribution-family subjects (Table IV), all detected; the `program_register_shape` static check flags the eighth—`qiskit_github_12`, the `measure_all` defect previously excluded for lacking a bijective map—and the ninth (a QPE/inverse-QFT case) awaits a sound relation. QMTes-ter

TABLE III

CANONICAL RESULTS, REGENERATED FROM THE FROZEN RUN’S `CANONICAL_RESULTS.CSV`. INTERVALS ARE WILSON 95% CIs. MORPHQ’S PROGRAM-LEVEL DETECTIONS ARE 0 BY THE TESTING LEVEL (SEC. III-E). [†]COVERAGE-BY-CONSTRUCTION (ADMIT-IF-DETECTED), NOT A DETECTION RATE (SEC. V-B). [§]THE 3 MINED SUBJECTS (§IV-B) MEET THE SAME CHECKS BUT LIE OUTSIDE THE FROZEN RUN. [‡]DEGENERATE CELL (MATCHED SEEDS GIVE $p \equiv 1$); THE NOISY/CALIBRATED ROWS ARE INFORMATIVE.

Cohort	Benchmark	Method	Metric	Count	Rate [95% CI]
Audited Bugs4Q	Program-level defect subset	QMTester	Detected	7/7	1.000 [0.646, 1.000]
Real defects (all) [§]	Bugs4Q + 3 mined (QCSE)	QMTester	Detected	10/10	1.000 [0.722, 1.000]
Audited Bugs4Q	Program-level defect subset	MorphQ	Detected	0/7	0.000 [0.000, 0.354]
Program injected [†]	100-case positive-control	QMTester	Detected	100/100	coverage-by-constr.
Program injected	100-case positive-control	MorphQ	Detected	20/100	0.200 [0.133, 0.289]
Fixed variants	7 Bugs4Q + 100 injected fixed	QMTester	False pos.	0/107	0.000 [0.000, 0.035]
Correct programs [‡]	ideal / noisy / calibrated sim	QMTester	False pos.	0/50 <i>each</i>	0.000 [0.000, 0.071]

thus *handles* 8 of the 9 admitted defects (7/7 on the distribution families) and flags none of the matched fixed variants. Beyond Bugs4Q, mining recent Quantum Computing StackExchange (2022–2024) adds *three* verified real defects (two QFT-round-trip, one input-permutation), each held to the *same* bar—buggy detected, matched fix clean (`program_mined_manifest`). The real distribution base is thus *ten*, all detected at 0 false positives (Wilson [0.722, 1.000], up from [0.646, 1.000]). The matched baselines extend: the reference-oracle detects all three (so QMTester matches the ceiling with *no* oracle on all *ten*), the static analyzer none (0/10); a circuit-level diagnostic incidentally fires on the QPE via swap-sensitivity (not a builder-contract detection, §IV-G).

This is detection on the relation-applicable subset, not a suite-wide rate. Three matched baselines (Table VI): MorphQ is structurally blind (0); the *deployable* static analyzer (LintQ-style register-shape check) also detects 0—the circuits being equally well-formed, it flags only the one *syntactic* `measure_all` case (covered by `program_register_shape`); a reference-oracle that *has* each known-correct version detects all, the ceiling but not deployable. QMTester reaches that ceiling with *no* oracle; MorphQ’s 0 is circuit-level blindness—structural on the builder-only families, a catalog gap on the two circuit-expressible ones (§III-E). We are plain about what the statistics do *not* carry: detection on all ten (deterministic) real subjects is carried by canonicalization (canonicalize-plus-raw-equality reaches 10/10, §IV-C). The permutation test and Bonferroni earn their keep on *specificity* and `parameter_periodicity`, the one probabilistic family, which still has *no real subject* (only the synthetic power curve, Table VII).

C. RQ2: Injected Positive Control

Sanity check (coverage-by-construction). Does each family fire on a balanced injected control, and what does QMTester’s machinery add over a trivial rebuild-and-compare?

Because a mutant is admitted here only if detected (§V-B), the 100/100 is coverage-by-construction (Table III): a sanity check that each family fires on its own contract, *not* a detection rate or contest. We retain it only as the within-method ablation substrate below. (MorphQ detects 20/100—*injected* single-circuit `ancilla_uncompute` mutants whose garbage is

circuit-visible—but *none* of the three *real* cross-boundary ancilla subjects, so its blindness holds for the real defects, not the injected proxy.)

Oracle-light baselines. To isolate QMTester’s machinery, three *program-level* baselines share the rebuild but each strip one component (Table VI): *same input* (no transform), *no canonicalization* (raw supports to the oracle), and *raw equality*—same seven subjects and 100 mutants, identical shots/seed/correction.

The result isolates two contributions. First, the metamorphic transform is necessary: *same input* detects 0/7 and 0/100. Second, canonicalization buys specificity: dropping it makes *no canonicalization* and *raw equality* false-positive on correct variants (2/7, 40/100) and miss buggy ones (5/7, 60/100 detected), the un-undone representation change both masquerading as a fault and masking real divergences. The real-subject differential is exactly the two `classical_remap` cases (2 points); the broader, inclusion-independent signal is the 40/100 injected fixed-variant false positives QMTester’s full pipeline holds at 0—within-method evidence MorphQ cannot provide.

D. RQ3: False Positives

What false-positive rate does QMTester report?

Specificity is the only inclusion-*independent* evidence in our evaluation, so we report it carefully. Across the 107 fixed program variants (7 Bugs4Q fixed subjects, 100 injected fixed variants) and the 50-program correct corpus, QMTester raises 0 false positives under the ideal, noisy, and calibrated-noisy simulators (Table III). The *ideal* cell is degenerate (matched seeds make source and follow-up bit-identical, $p \equiv 1$); the informative cells are the noisy ones. Under the bundled depolarizing model ($p_1=10^{-3}$, $p_2=5 \times 10^{-3}$, $T_1=T_2=80 \mu\text{s}$, $t_g=50 \text{ ns}$, density-matrix sim; `noise.json`) the smallest *uncorrected* per-pair p over all 3233 pairs is 0.370 (noisy) and 0.251 (calibrated)—conservative, but not by a wide margin. To stress this we re-ran the corpus under a *realistic* model adding 2% symmetric readout error and a 1% two-qubit crosstalk term: the smallest uncorrected p falls to 0.0035 (below the nominal 0.05), yet source-level Bonferroni still yields 0/50. So under realistic noise the per-pair margin is genuinely thin and the multiple-testing correction—not merely conservative nulls—is what holds specificity, a qualifier the four 0/50 rows

TABLE IV

AUDITED BUGS4Q PROGRAM-LEVEL SUBJECTS (IN-SCOPE). A SUBJECT IS *included* ONLY WHEN A SOUND PROGRAM-LEVEL RELATION CAN BE DOCUMENTED (SOURCE→FOLLOW-UP INPUT TRANSFORMATION, BIJECTIVE CANONICALIZATION MAP, AND A SOUNDNESS ARGUMENT). FOR EACH, THE BUGGY VARIANT IS DETECTED AND THE FIXED VARIANT IS CLEAN. FULL PER-SUBJECT AUDIT (SOURCE/FOLLOW-UP INPUTS, CANONICAL MAPS, SOUNDNESS ARGUMENTS, OBSERVED VIOLATIONS) IS IN THE RELATION-SOUNDNESS AUDIT BUNDLED WITH THE ARTIFACT.

Subject	Family (program_*)	Metamorphic transform (source → follow-up input)	Buggy	Fixed
github_17_meas_order	classical_remap	Remap classical-register read order; canonicalize back to source layout.	detected	clean
so_5_ccx_role	input_permutation	Permute the two symmetric CCX control roles; invert on output.	detected	clean
so_1_teleport_feedback	ancilla_uncompute	Require clean teleportation feedback/ancilla state under declared uncompute.	detected	clean
se_3_ccx_uncompute	ancilla_uncompute	Complete the declared CCX ancilla uncompute; logical output invariant.	detected	clean
se_12_meas_endian	classical_remap	Flip classical-register endianness; canonicalize to source order.	detected	clean
so_7_qft_direction	qft_round_trip	Compose QFT with its declared inverse; round trip is identity on input.	detected	clean
se_8_grover_uncompute	ancilla_uncompute	Require clean Grover ancilla after the declared uncompute.	detected	clean
Included denominator: 7 QMTester 7/7 detected MorphQ 0/7 fixed variants 0/7 false positives				

TABLE V

DISPOSITION OF *all* 32 BUILD-AND-RUN BUGS4Q BUGS (FULL AUDIT; PER-BUG DETAIL IN THE ARTIFACT). NINE ADMIT A SOUND BUILDER-CONTRACT RELATION—EIGHT NOW DETECTED (SEC. IV-B)—AND 23 ARE OUT OF SCOPE BY THE CATEGORIES BELOW. WITH EVERY BUG DISPOSITIONED, THE IN-SCOPE COUNT IS A POPULATION PROPERTY, NOT AN AUDIT-COVERAGE ARTIFACT.

Out-of-scope category	Count
API misuse (deprecated API, wrong import/attribute)	12
Output-formatting (visualization, global phase)	4
Platform/runtime (backend, transpiler, version)	3
Generic wrong-gate / missing-gate	3
Measurement is not a program contract for this bug	1
Out of scope: 23 of 32 (in scope: 9; detected: 8)	

alone would hide. Concretely, the thin-margin subject (smallest realistic-noise $p = 0.0035$) carries $m = 96$ admitted pairs, so $\alpha/m = 0.0005 \leq 0.0035$; across the corpus $m \in [30, 96]$, every per-subject threshold below its subject’s smallest margin, so 0/50 holds by correction, not by the thin-margin pair landing on a high- m subject by luck.

E. Statistical-Oracle Behavior

We verify two properties of the statistical oracle directly from the logged per-pair records. *Empirical type-I*: across the 50 correct programs, none of the 3233 per-pair null p-values falls below 0.05 even uncorrected under any of the three simulators (minimum 1.000/0.370/0.251 for ideal/noisy/calibrated-noisy). The nulls are thus not uniform but *conservative*, the favorable direction for Remark 1, matching the observed 0/107 and 0/50 false-positive rates. *Which branch decides*: the Monte-Carlo permutation branch, not the asymptotic χ^2 , fires for most program-level decisions (85.7% of Bugs4Q and 93.0% of injected pairs), as the small quantum supports never satisfy the Cochran asymptotic condition (zero merges across all admitted pairs); we therefore describe the oracle as permutation-primary with an asymptotic χ^2 fast path.

F. RQ4: Sensitivity Floor (Detection vs. Effect Size)

At what effect size does detection start to fail?

TABLE VI

PROGRAM-LEVEL BASELINES (DETECTION ON BUGGY/MUTANT, FP ON MATCHED FIXED; 4096 SHOTS, IDEAL SIM). THE REFERENCE-ORACLE DIFFERENTIAL IS A NON-DEPLOYABLE DETECTION CEILING; THE *deployable* LINTQ-STYLE REGISTER-SHAPE CHECK CATCHES ONLY THE *syntactic* MEASURE_ALL CASE, NONE OF THE 7 *semantic* DEFECTS. INDENTED ROWS ARE ORACLE-LIGHT ABLATIONS: CANONICALIZE-PLUS-RAW-EQUALITY ALONE ALREADY GIVES 7/7, SO THE PERMUTATION TEST SERVES SPECIFICITY, NOT DETECTION.

Method	Det.	FP
<i>Audited Bugs4Q defects (7 buggy / 7 fixed)</i>		
MorphQ (circuit-level; blind)	0/7	—
Static analyzer (<i>deployable</i> ; LintQ-style)	0/7	0/7
Reference-oracle (external; <i>has</i> oracle)	7/7	0/7
QMTester (full; oracle-free)	7/7	0/7
canonicalize + raw equality (no test)	7/7	0/7
same input + χ^2 (no transform)	0/7	0/7
no canonicalization + χ^2	5/7	2/7
raw count-equality, no canon.	5/7	2/7
<i>Injected benchmark (100 mutant / 100 fixed)</i>		
QMTester (full)	100/100	0/100
same input + χ^2	0/100	0/100
no canonicalization + χ^2	60/100	40/100
raw count-equality, no canon.	60/100	40/100

The families differ in output character. `input_permutation`, `classical_remap`, `ancilla_uncompute`, and the real `qft_round_trip` subject all produce *deterministic* basis-state divergence (TVD= 1.0); the 0.74–0.94 figure is the input-dependent effect size of the *injected* QFT mutants, not the real subject. Only the probabilistic `parameter_periodicity` family has a tunable effect. A 204-mutant rotation-drift sweep (TVD $\in$ (0, 0.06), *not* filtered by detectability; true effect at 32,768 shots, detection at 4096) localizes the threshold via a fine sub-0.03 grid (Table VII): the 50%/80% crossings fall in [0.022, 0.025]/[0.025, 0.030). So the conservative nulls do not silently trade away sensitivity—the method detects reliably above a measured floor near TVD 0.025 (probabilistic family only).

TABLE VII
POWER CURVE (DETECTION RATE VS. EFFECT SIZE) FOR
PROGRAM_PARAMETER_PERIODICITY AT 4096 SHOTS, OVER A
204-MUTANT ROTATION-DRIFT SWEEP OVER TVD $\in (0, 0.06)$, *not*
FILTERED BY DETECTABILITY (TRUE EFFECT AT 32,768 SHOTS). THE FINE
SUB-0.03 SWEEP *localizes* THE THRESHOLD: 50% IN $[0.022, 0.025)$, 80%
IN $[0.025, 0.030)$. OTHER FOUR FAMILIES: (NEAR-)DETERMINISTIC
(TVD ≈ 1), NO SMALL-EFFECT REGIME.

Effect size (TVD)	Detected	Rate (Wilson 95% CI)
[0.000, 0.018)	12/155	0.08 [0.04, 0.13]
[0.018, 0.022)	5/12	0.42 [0.19, 0.68]
[0.022, 0.025)	7/10	0.70 [0.40, 0.89]
[0.025, 0.060)	27/27	1.00 [0.88, 1.00]

G. Diagnostic Circuit-Level Run

For transparency, a circuit-level-only configuration (identity/swap/phase/equivalence rewritings) shows *no* advantage over MorphQ—2/32 on Bugs4Q, 17/250 injected, identical for both, all from swap—motivating the program-level design (not a contribution claim).

V. DISCUSSION

The *level* at which a relation is defined determines which bugs it sees: a relation over the final circuit takes the buggy output as ground truth (our diagnostic confirms no advantage over MorphQ), whereas rebuilding from a follow-up input lets a divergence reflect a contract violation, not a finite-shot artifact.

A. Artifact Verifiability

Every headline number regenerates from a canonical CSV that `derive_metrics.py` builds from run-isolated summary shards under a frozen run directory, with denominator enforcement and a re-derivation integrity check. The artifact bundles the adapters, relation families, canonicalization, the pipeline, all manifests (Bugs4Q core + `program_mined`, held to the same admission/baseline checks), the soundness audit, MorphQ logs, and derivation scripts.

B. Threats to Validity

Small, load-bearing real-defect denominator. The distribution base is *ten* subjects (seven Bugs4Q + three mined, §IV-B; eleven with register-shape). The class is rare but *not* exhausted: static-finding datasets yield little (LintQ [30], one distribution-mappable finding + the 37 `measure_all` changes; OOPSLA’22 mostly off-contract [31]), whereas Q&A mining returns about one genuine defect per four candidates. The new subjects cluster in deterministic families; a real `parameter_periodicity` defect remains open—and the gap is *principled*. The dominant real periodicity bug is a *period confusion* (treating a controlled rotation as 2π -periodic when its period is 4π , documented in Qiskit itself [25]); QMTester catches this at *admission*—rejecting the unsound 2π relation—not via the distribution test, since testing it would flag a *correct* program (the false positive a naive MR raises). The distribution-detectable alternative, a wrong-*scaling* angle error,

an exhaustive Q&A screen finds vanishingly rare. So this family’s statistics are validated on the synthetic sweep, with the period-aware admission its real-world anchor [25].

Construct and external validity. Preconditions are auditable (Table II; A_R logged per subject/family). The evaluation is Qiskit/simulator-based, but we *measure* the noise question: under the realistic +2% readout/+1% crosstalk model the seven Bugs4Q defects stay detected (mined three not re-run) and the basis-state families specific, while one cross-boundary ancilla fixed variant trips a false positive (most noise-sensitive). The relations are also *not Qiskit-specific*: rebuilding two real subjects (role permutation, register remapping) in *Cirq* under the *same* canonicalization and oracle, the buggy variant is detected and the fixed clean (`run_cirq_crossstack`)—only circuit construction is stack-bound. Proposition 1 is noise-independent; device and Q# runs remain untested.

C. Limitations and Future Work

Mining recent Q&A is the productive lever; scaling it, with further relation families (expectation-value, Pauli-string), is the path to a larger base—recent Qiskit (1.x/2.x) the richest source.

VI. CONCLUSION

QMTester defines metamorphic relations at the *program/builder* level; its contribution is *admission by canonicalization*—not the input transform, which is standard metamorphic testing. It *rejects* pre-execution a follow-up whose output map is not a documented bijection—a *systematic* bijectivity-on-measured-support guard (a register-dropping relabel a naive MR misreads as a fault and an ad-hoc author only sometimes catches)—so a divergence reads as a contract violation. On ten real subjects it detects 10/10—matching a reference-oracle on the seven Bugs4Q with *no* oracle—at 0 false positives. The detectable class is rare but is exactly the one no MorphQ-class transform recovers (Proposition 1): the discriminating *contract*—roles, endianness, uncompute, the reusable lever—is recoverable only by rebuilding.

REFERENCES

- [1] J. Preskill, “Quantum computing in the NISQ era and beyond,” *Quantum*, vol. 2, p. 79, 2018.
- [2] Y. Huang and M. Martonosi, “Statistical assertions for validating patterns and finding bugs in quantum programs,” in *Proceedings of the 46th International Symposium on Computer Architecture*. ACM, 2019, pp. 541–553.
- [3] G. Li, L. Zhou, N. Yu, Y. Ding, M. Ying, and Y. Xie, “Projection-based runtime assertions for testing and debugging quantum programs,” *Proceedings of the ACM on Programming Languages*, vol. 4, no. OOPSLA, pp. 150:1–150:29, 2020.
- [4] P. Zhao, Z. Miao, S. Lan, and J. Zhao, “Bugs4Q: A benchmark of existing bugs to enable controlled testing and debugging studies for quantum programs,” *Journal of Systems and Software*, vol. 205, p. 111805, 2023.
- [5] M. Paltenghi and M. Pradel, “MorphQ: Metamorphic testing of the Qiskit quantum computing platform,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2413–2424.

- [6] J. Wang, Q. Zhang, G. H. Xu, and M. Kim, "QDiff: Differential testing of quantum software stacks," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2021, pp. 692–704.
- [7] N. H. Oldfield, C. Laaber, T. Yue, and S. Ali, "Faster and better quantum software testing through specification reduction and projective measurements," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 7, pp. 1–39, 2025.
- [8] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: A new approach for generating next test cases," Department of Computer Science, Hong Kong University of Science and Technology, Tech. Rep. HKUST-CS98-01, 1998.
- [9] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.
- [10] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortes, "A survey on metamorphic testing," *IEEE Transactions on Software Engineering*, vol. 42, no. 9, pp. 805–824, 2016.
- [11] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys*, vol. 51, no. 1, pp. 4:1–4:27, 2018.
- [12] H. Liu, F.-C. Kuo, D. Towey, and T. Y. Chen, "How effectively does metamorphic testing alleviate the oracle problem?" *IEEE Transactions on Software Engineering*, vol. 40, no. 1, pp. 4–22, 2014.
- [13] D. Fortunato, J. Campos, and R. Abreu, "QMutPy: a mutation testing tool for quantum algorithms and applications in Qiskit," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2022, pp. 797–800.
- [14] A. Muqeet, S. Ali, and P. Arcaini, "Quantum program testing through commuting Pauli strings on IBM's quantum computers," arXiv preprint arXiv:2408.00501, 2024. [Online]. Available: <https://arxiv.org/abs/2408.00501>
- [15] A. Muqeet, S. Ali, and P. Arcaini, "Search-based quantum program testing via commuting Pauli string," arXiv preprint arXiv:2602.11487, 2026. [Online]. Available: <https://arxiv.org/abs/2602.11487>
- [16] J. Luo, S. Xia, F. Zhang, and J. Zhao, "QEMI: A quantum software stacks testing framework via equivalence modulo inputs," arXiv preprint arXiv:2602.09942, 2026. [Online]. Available: <https://arxiv.org/abs/2602.09942>
- [17] R. Abreu, J. P. Fernandes, L. Llana, and G. Tavares, "Metamorphic testing of oracle quantum programs," in *Proceedings of the 3rd International Workshop on Quantum Software Engineering (Q-SE)*, 2022, pp. 16–23.
- [18] N. Costa, J. P. Fernandes, and R. Abreu, "Asserting the correctness of shor implementations using metamorphic testing," in *Proceedings of the 1st International Workshop on Quantum Programming for Software Engineering (QP4SE)*, 2022, pp. 32–36.
- [19] L. J. Kitt and M. B. Cohen, "MorphQ++: A reproducibility study of metamorphic testing on quantum compilers," in *Proceedings of the 2024 Workshop on Replications and Negative Results (RNR)*, 2024, pp. 8–14.
- [20] M. Ying, "Floyd–Hoare logic for quantum programs," *ACM Transactions on Programming Languages and Systems*, vol. 33, no. 6, pp. 19:1–19:49, 2011.
- [21] D. Unruh, "Quantum relational Hoare logic," *Proceedings of the ACM on Programming Languages*, vol. 3, no. POPL, pp. 33:1–33:31, 2019.
- [22] J. Paykin, R. Rand, and S. Zdancewic, "QWIRE: A core language for quantum circuits," in *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*, 2017, pp. 846–858.
- [23] T. Peham, L. Burgholzer, and R. Wille, "Equivalence checking of quantum circuits with the ZX-calculus," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 12, no. 3, pp. 662–675, 2022.
- [24] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge, United Kingdom: Cambridge University Press, 2010.
- [25] Qiskit contributors, "Fix 4π periodicity of controlled rotations in the CommutationChecker," Qiskit pull request #13670, 2025, <https://github.com/Qiskit/qiskit/pull/13670>.
- [26] K. Pearson, "On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 50, no. 302, pp. 157–175, 1900.
- [27] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [28] A. Muqeet, T. Yue, S. Ali, and P. Arcaini, "Mitigating noise in quantum software testing using machine learning," *IEEE Transactions on Software Engineering*, vol. 50, no. 11, pp. 2947–2961, 2024.
- [29] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
- [30] M. Paltenghi and M. Pradel, "Analyzing quantum programs with LintQ: A static analysis framework for Qiskit," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, 2024.
- [31] M. Paltenghi and M. Pradel, "Bugs in quantum computing platforms: An empirical study," *Proceedings of the ACM on Programming Languages*, vol. 6, no. OOPSLA1, 2022.